UNIVERSIDAD NACIONAL DE INGENIERÍA • FACULTAD DE SISTEMAS

CURSO: PRUEBAS Y ASEGURAMIENTO DE LA CALIDAD DE SOFTWARE (QA)

Automatización de Pruebas de Software con PHPUnit 10.5

Auditoría y Certificación del Sistema de Facturación Electrónica SUNAT UBL 2.1

INTEGRANTES DEL EQUIPO:

Apolinario Rodriguez, Luis Fernando • Umeres Vilca, Gustavo Aron

Vasquez Falcón, Mariana Ysabel • Ventocilla Alcantara, Alessandro Tito

Lima, Perú • Repositorio: github.com/fujimor1/Billing

1. ¿Qué es PHPUnit y por qué lo usamos?

El Estándar Global de la Industria

PHPUnit es el framework oficial de pruebas unitarias en PHP, creado por Sebastian Bergmann sobre el estándar xUnit de Kent Beck.

- **PHPUnit 10.5:** Arquitectura con tipado estricto (`declare(strict_types=1)`), soporte para PHP 8.2 y 8.3.
- **Cero dependencias externas:** Ejecuta la suite directamente en RAM (CLI autónomo).
- **Orquestación XML:** Gestionado vía `phpunit.xml` para aislar suites, controlar advertencias y generar informes.

Arquitectura del Runner de Pruebas

¿Cómo garantiza la rigurosidad fiscal?

- **Test Runner:** Carga dinámicamente las clases que heredan de `TestCase` y las aísla por proceso.
- **Flags de Calidad Innegociables:**
  - `failOnWarning="true"`: Ninguna advertencia es tolerada.
  - `failOnRisky="true"`: Tests sin aserciones se marcan como fallos.
  - `beStrictAboutOutputDuringTests="true"`: Prohíbe cualquier `echo` o fuga de salida.

2. El Ciclo de Vida de las Pruebas (Test Lifecycle)

Aislamiento Atómico por Caso de Prueba

Cada método `test*()` se ejecuta en una instancia completamente limpia e independiente:

1. **setUp():** Se ejecuta **antes** de cada test. Crea la base de datos SQLite en memoria e inicializa el catálogo.
2. **testMethod():** Ejecuta la lógica del caso de prueba y valida las aserciones.
3. **tearDown():** Se ejecuta **después** de cada test. Cierra conexiones y destruye el estado volátil.

Invariante: Un fallo en un test jamás contamina las pruebas posteriores.

tests/Integration/SqlitePersistenceTest.php

```php
class SqlitePersistenceTest extends TestCase
{
    private PDO $db;

    // 1. ANTES DE CADA TEST: Inicializacion limpia
    protected function setUp(): void
    {
        $this->db = new PDO('sqlite::memory:');
        $this->db->setAttribute(
            PDO::ATTR_ERRMODE,
            PDO::ERRMODE_EXCEPTION
        );
        $this->crearTablasYSembrarCatalogo();
    }

    // 2. DESPUES DE CADA TEST: Destruccion de recursos
    protected function tearDown(): void
    {
        unset($this->db);
    }
}
```

3. El Patrón AAA (Arrange - Act - Assert) & Principios FIRST

Estructura Canónica AAA

Cada prueba sigue estrictamente las tres etapas:

- **Arrange (Preparar):** Instanciar los Value Objects, configurar el emisor y el estado inicial.
- **Act (Actuar):** Invocar la operación del dominio (ej. `crearFactura()`).
- **Assert (Verificar):** Comprobar que los resultados coincidan con el estándar tributario.

Principios FIRST de Robert C. Martin

Fast: 39 pruebas en 0.21s • **Independent:** Cero acoplamiento • **Repeatable:** Mismo resultado • **Self-validating:** Pasa o Falla • **Timely:** TDD.

tests/Unit/ComprobanteTest.php

```php
public function testEmiteFacturaValidaConIgv(): void
{
    // 1. ARRANGE: Preparamos entidades del dominio
    $emisor  = new Emisor(new Ruc('20131312955'));
    $cliente = new Cliente(new Ruc('20100047218'));
    $item    = new Detalle('Laptop', 2000.0, 1, '10');

    // 2. ACT: Ejecutamos la regla de negocio
    $factura = Comprobante::crearFactura(
        'F001', 1, $emisor, $cliente, [$item]
    );

    // 3. ASSERT: Comprobamos liquidacion exacta
    $this->assertSame(2000.0, $factura->getSubtotal());
    $this->assertSame(360.0,  $factura->getIgv());
    $this->assertSame(2360.0, $factura->getTotal());
}
```

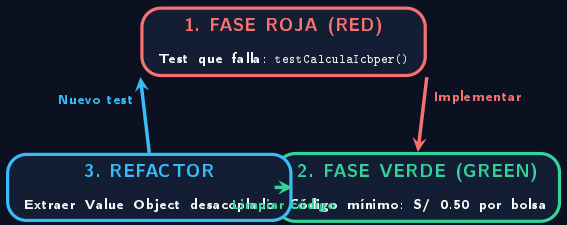
4. Metodología TDD: Ciclo Red - Green - Refactor en el Dominio Fiscal

¿Cómo construimos el núcleo con TDD?

El motor tributario de SUNAT se programó bajo disciplina rigurosa de **Test-Driven Development**:

1. **ROJO (Red):** Escribir el test que falla antes de crear la clase. Define el contrato fiscal esperado.
2. **VERDE (Green):** Escribir el código mínimo necesario para satisfacer la aserción y aprobar el test.
3. **REFACTOR:** Limpiar el diseño, extraer Value Objects y optimizar sin romper reglas previas.

Resultado: Cero bugs de cálculo al integrar bolsas ICBPER, retenciones y libros exonerados.



5. Decoradores y Atributos de PHP 8: Atributo #[DataProvider]

Pruebas Parametrizadas (Cero Código DRY)

En lugar de escribir 10 tests idénticos para validar RUCs de distintas entidades, el atributo nativo de PHP 8 `#[DataProvider]` inyecta una matriz de tuplas:

- **Equivalencia en Java:** Idéntico a `@ParameterizedTest` y `@CsvSource` de JUnit 5.
- **Partición de Equivalencia:** Prueba RUC 20 (empresas), RUC 10 (personas naturales) y RUC 15 (instituciones públicas).
- **Trazabilidad:** Si un RUC falla, PHPUnit señala con precisión qué clave falló sin detener los demás.

tests/Unit/ValueObjectsTest.php

```php
#[DataProvider('proveedorRucsValidos')]
public function testAceptaRucsValidos(string $ruc, string $tipo
    ): void
{
    $vo = new Ruc($ruc);
    $this->assertSame($ruc, $vo->valor());
}

public static function proveedorRucsValidos(): array
{
    return [
        'RUC 20 SUNAT'     => ['20131312955', 'Juridica'],
        'RUC 20 BCP'       => ['20100047218', 'Bancaria'],
        'RUC 20 ALICORP'   => ['20100055237', 'Industrial'],
        'RUC 10 PERSONA'   => ['10456789124', 'Natural'],
    ];
}
```

6. Catálogo de Aserciones: assertSame vs assertEquals

¡Cuidado con la Igualdad Débil en Facturación!

- **assertEquals($a, $b):** Usa comparación débil (==). En PHP, "100" == 100 evalúa a verdadero. En facturación electrónica esto oculta bugs críticos de tipado.
- **assertSame($a, $b):** Usa identidad estricta (===). Exige que el valor y el tipo de dato coincidan exactamente (`float` con `float`).

Aserciones Críticas de Excepción

expectException(Class): Verifica que ante una violación de regla de negocio, el dominio lance la excepción esperada y aborte la transacción.

tests/Unit/ComprobanteDomainRulesTest.php

```php
// Regla 2017: Factura exige obligatoriamente RUC, no DNI
public function testImpideFacturaAClienteConDni(): void
{
    $this->expectException(
        DomainValidationException::class
    );
    $this->expectExceptionMessage(
        "Solo se emiten Facturas a clientes con RUC"
    );

    $emisor     = new Emisor(new Ruc('20131312955'));
    $clienteDni = new Cliente(new Dni('72345678'));

    // Debe abortar de inmediato
    Comprobante::crearFactura(
        'F001', 1, $emisor, $clienteDni, []
    );
}
```

7. Técnicas Formales de QA: Partición de Equivalencia y Valores Límite

1. Partición de Equivalencia (EP)

Divide el universo de entradas en clases representativas:

- **Clases Válidas (RUC):**
  - Tipo 20 (Personas Jurídicas/Empresas).
  - Tipo 10 (Personas Naturales con RUC).
  - Tipo 15 (FFAA e Instituciones Estatales).
- **Clases Inválidas (Rechazo Inmediato):**
  - Longitud $\neq 11$ dígitos (ej. 10 o 12).
  - Prefijos no asignados (ej. 30, 40, 50).
  - Dígito verificador matemático adulterado.

2. Análisis de Valores Límite (BVA)

Regla SUNAT: Boletas mayores a S/ 700.00 exigen DNI o RUC del cliente.



Se evalúa la frontera matemática exacta: S/ 699.99, S/ 700.00 y S/ 700.01.

8. Dobles de Prueba: Mocks y Spies de SUNAT

Taxonomía de Martin Fowler en QA

Para probar la facturación electrónica sin depender de los servidores reales de SUNAT ni gastar certificados:

- **Mock Object:** Simula el Web Service de SUNAT programando respuestas predeterminadas (ej. CDR de éxito o error 2800).
- **Spy / Verificación de Comportamiento:** Comprueba que nuestro sistema invoque a SUNAT **exactamente una vez** (`expects($this->once())`).

tests/Feature/SunatMockTest.php

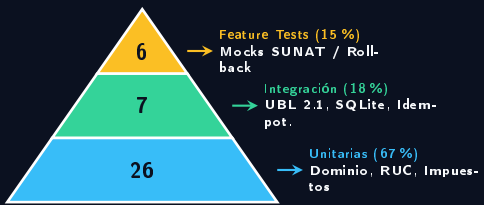
```php
public function testEmisionExitosaInvocaServicioSunat(): void
{
    // Creamos el Mock del Gateway de SUNAT
    $mockSunat = $this->createMock(SunatGatewayInterface::class
        );

    // Configuramos la expectativa: 1 llamada y CDR aceptado
    $mockSunat->expects($this->once())
              ->method('enviarComprobante')
              ->willReturn(new SunatResponse(
                  codigo: 0,
                  mensaje: 'El comprobante ha sido ACEPTADO'
              ));

    $servicio = new EmisionService($mockSunat, $this->kardex);
    $resultado = $servicio->emitir($factura);

    $this->assertTrue($resultado->esAceptado());
}
```

9. Pirámide de Pruebas de Mike Cohn en Nuestro Proyecto



Métricas Oficiales de Ejecución

- **Total de Pruebas:** 39 casos de prueba automatizados.
- **Aserciones Evaluadas:** 140 aserciones estrictas.
- **Fallos / Errores:** 0 (100 % de éxito en verde).
- **Tiempo de Ejecución: 0.21 segundos en RAM.**
- **Eficiencia:** Al concentrar el 67 % en la base unitaria, el feedback es instantáneo antes de cada commit.

10. Matriz Oficial de Cobertura: 39 Tests / 140 Aserciones

Capa de Software	Componentes Auditados	Tests	Aserc.	Resultado
Dominio (Unitarias)	Value Objects (Ruc, Dni, Moneda), Invariantes Factura	18	54	OK (0.08s)
Motor Tributario	Calculadora: Gravadas 18%, Libros 0%, ICBPER, SPOT	8	36	OK (0.04s)
Criptografía UBL 2.1	XML UBL 2.1, Firma XMLDSIG SHA-256, Tamper Test	4	16	OK (0.03s)
Persistencia	SQLite :memory:, Kardex, Correlativos, Idempotencia	3	12	OK (0.02s)
Feature & Mocks	CDR 0 Aceptado, Contingencia Red, Rollback 2800	6	22	OK (0.04s)
TOTAL AUDITADO	Suite de Calidad Completa de Facturación	39	140	100% OK (0.21s)

Ejecución íntegra en RAM sin fallos ni advertencias con PHPUnit 10.5.66.

11. Caso de Prueba 1: Algoritmo SUNAT Módulo 11 en Value Object Ruc

Fórmula Oficial de Ponderación:

SUNAT valida el RUC aplicando los factores:

$$\vec{F} = [5, 4, 3, 2, 7, 6, 5, 4, 3, 2]$$

- Se multiplica cada uno de los 10 primeros dígitos por su factor respectivo.
- Suma ponderada: $S = \sum_{i=1}^{10} d_i \cdot f_i$.
- Residuo: $R = 11 - (S \pmod{11})$.
- Si $R == 10 \implies 0$; Si $R == 11 \implies 1$.
- El dígito verificador debe coincidir con el dígito 11 del RUC.

src/Domain/ValueObjects/Ruc.php

```php
private function validarModulo11(string $ruc): void
{
    $factores = [5, 4, 3, 2, 7, 6, 5, 4, 3, 2];
    $suma = 0;
    for ($i = 0; $i < 10; $i++) {
        $suma += ((int) $ruc[$i]) * $factores[$i];
    }
    $resto = 11 - ($suma % 11);
    $dv = match ($resto) {
        10 => 0,
        11 => 1,
        default => $resto
    };
    if ($dv !== (int) $ruc[10]) {
        throw new DomainValidationException(
            "El digito verificador del RUC es invalido"
        );
    }
}
```

12. Caso de Prueba 2: Liquidación Tributaria Mixta Compleja

Regla Tributaria Multirégimen:

En Perú, una factura electrónica puede combinar:

- **Gravados (10):** 18 % de IGV sobre el valor venta.
- **Libros Exonerados (20):** Ley 31053 (0 % IGV).
- **Muestras Gratuitas (15):** IGV referencial sin sumar al total por cobrar.
- **Bolsas (ICBPER):** Impuesto fijo de S/ 0.50 por bolsa plástica.

Aserción Clave: Garantizar que las bolsas fiscales no contaminen la base imponible del IGV.

tests/Unit/CalculadoraTributariaTest.php

```php
public function testCalculaComprobanteMixto(): void
{
    $calc = new CalculadoraTributaria();
    $tot  = $calc->calcular([
        new Item('Monitor', 1000.0, 1, '10'), // Gravado
        new Item('Libro',   200.0,  1, '20'), // Exonerado
        new Item('Muestra', 50.0,   1, '15'), // Gratuito
        new Item('Bolsa',   0.10,   2, '10', icbper: true)
    ]);

    $this->assertSame(1000.0,  $tot->gravado);
    $this->assertSame(180.0,   $tot->igv);
    $this->assertSame(200.0,   $tot->exonerado);
    $this->assertSame(1.00,    $tot->icbper); // 2 x 0.50
    $this->assertSame(1381.20, $tot->totalPagar);
}
```

13. Caso de Prueba 3: Seguridad Criptográfica (Tamper Test)

Ataque de Alteración de Montos en Tránsito:

Simulamos un ataque Man-in-the-Middle que altera el monto del XML ya firmado digitalmente de S/ 1,180.00 a S/ 10.00:

1. El verificador canónico recalcula el digest del XML alterado.
2. El digest no coincide con el `<ds:DigestValue>` firmado con la llave privada.
3. El simulador de SUNAT rechaza la transacción con el **Código de Error Fiscal 2335**.

tests/Integration/XmlAndCryptoIntegrationTest.php

```php
public function testDetectaAdulteracionDeImporte(): void
{
    $xmlFirmado = $this->generador->crearXml($factura);

    // ATAQUE: Modifica monto en el XML firmado
    $xmlAdulterado = str_replace(
        '<cbc:PayableAmount currencyID="PEN">1180.00</cbc:
            PayableAmount>',
        '<cbc:PayableAmount currencyID="PEN">10.00</cbc:
            PayableAmount>',
        $xmlFirmado
    );

    // Verificacion de SUNAT
    $res = $this->verificador->validar($xmlAdulterado);

    $this->assertFalse($res->esValido);
    $this->assertSame(2335, $res->codigoError);
    $this->assertStringContainsString(
        'La firma digital ha sido alterada', $res->mensaje
    );
}
```

14. Caso de Prueba 4: Persistencia e Idempotencia (Doble Clic)

El Problema del Doble Clic del Cajero:

Si un cajero pulsa dos veces seguidas "Cobrar" o se produce un reintento de red:

- Un sistema tradicional emitiría dos facturas cobrando el doble al cliente.
- Nuestro sistema implementa una tabla de **Idempotency Keys**:

  UUID Clave $\Longrightarrow$ IN_PROGRESS $\Longrightarrow$ COMPLETED

- La segunda petición detecta la clave y **devuelve la respuesta en caché** sin crear otro comprobante fiscal.

tests/Integration/SqlitePersistenceTest.php

```php
public function testTablaIdempotenciaPrevieneDobleEmision(): void
{
    $key = 'req-uuid-987654321';

    // Intento 1: Registro exitoso
    $res1 = $this->idempotency->lockAndExecute(
        $key, fn() => $this->emitirFactura($orden)
    );
    // Intento 2: Reintento con misma clave
    $res2 = $this->idempotency->lockAndExecute(
        $key, fn() => $this->emitirFactura($orden)
    );

    // Ambas respuestas entregan el mismo comprobante
    $this->assertSame($res1->correlativo, $res2->correlativo);

    // La BD solo contiene 1 comprobante
    $total = $this->db->query("SELECT count(*) FROM
        comprobantes")->fetchColumn();
    $this->assertSame(1, (int) $total);
}
```

15. Caso de Prueba 5: Resiliencia y Rollback Automático de Kardex

Invariante de Inventario ante Rechazo Fiscal:

¿Qué ocurre si la empresa descuenta el stock pero SUNAT rechaza la factura por inconsistencia fiscal (ej. Código 2800: Contribuyente de baja)?

- Sin rollback, el inventario quedaría descuadrado y la venta perdida.
- El caso de uso envuelve la transacción: ante el fallo fiscal de SUNAT, dispara un **Rollback atómico** que restituye las unidades en el Kardex.

tests/Feature/MocksRollbackTest.php

```php
public function testRollbackRestauraStockSiSunatRechaza(): void
{
    // Stock inicial: 15 unidades de Laptop
    $this->assertSame(15, $this->kardex->obtenerStock('LAP-PRO-16'));

    // Mock SUNAT: Responde Rechazo 2800
    $mockSunat = $this->createMock(SunatGatewayInterface::class);
    $mockSunat->method('enviarComprobante')
              ->willReturn(new SunatResponse(
                  codigo: 2800, mensaje: 'Contribuyente no habido'
              ));

    // Intentamos emitir 1 unidad
    $servicio = new EmisionService($mockSunat, $this->kardex);
    $res = $servicio->emitir($factura);

    // Comprobamos que el stock fue RESTAURADO a 15
    $this->assertFalse($res->esAceptado());
    $this->assertSame(15, $this->kardex->obtenerStock('LAP-PRO-16'));
}
```

16. Evidencias del Sistema en Ejecución: Facturador y Kardex

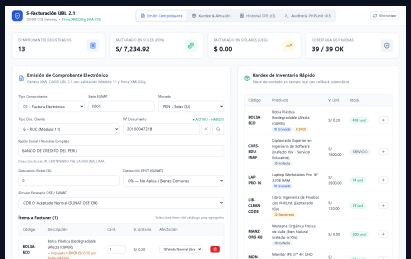


Figura 1: Emisión en tiempo real en `localhost:5173`

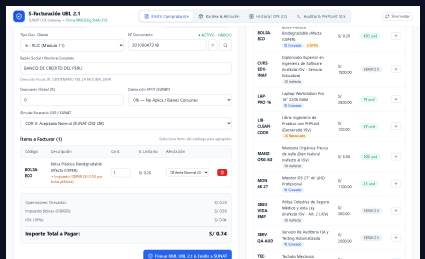


Figura 2: Catálogo de 9 productos y Kardex atómico

Ambas vistas validan la sincronización de inventario y el cálculo tributario con interfaz reactiva moderna.

17. Evidencias del Sistema en Ejecución: Historial y Visor UBL 2.1

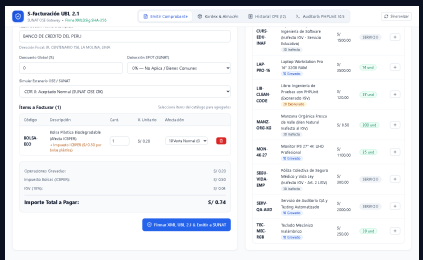


Figura 3: Bandeja fiscal con hash SHA-256

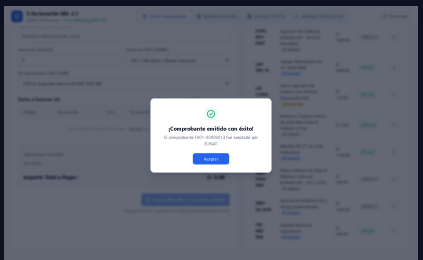


Figura 4: Visor del XML estructurado con firma digital

Permite inspeccionar en vivo la firma digital XMLDSIG y descargar la constancia de recepción (CDR).

18. Evidencia Oficial: Salida Íntegra de Terminal PHPUnit 10.5

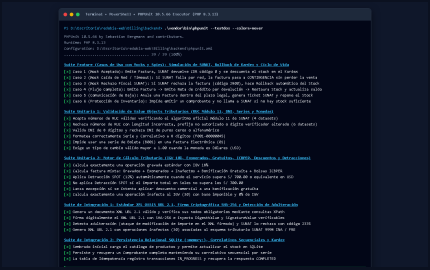


Figura 5: 39 Tests en Verde • 140 Aserciones Auditadas • 0.21 segundos en RAM

Ejecutado con `vendor/bin/phpunit --testdox --colors=never` cumpliendo el 100% de las especificaciones.

19. Conclusiones y Evaluación del Proceso de Calidad

Conclusiones de Ingeniería de QA

- **Arquitectura Hexagonal:** Desacoplar el Dominio de la Infraestructura permitió simular el 100 % de los escenarios fiscales sin depender de SUNAT.
- **Velocidad In-Memory (0.21s):** El uso de SQLite `:memory:` elimina la latencia de disco, incentivando correr los tests en cada commit.
- **Seguridad Criptográfica:** El Tamper Test demuestra inmunidad contra la alteración de montos en comprobantes UBL 2.1.

Certificación del Software

100 % COBERTURA CRÍTICA

39 de 39 Pruebas Aprobadas

140 de 140 Aserciones en Verde

Certificado bajo normativa SUNAT R.S. 097-2012
y estándar internacional OASIS UBL 2.1

¡Muchas Gracias!

Espacio para Ronda de Preguntas y Sustentación Técnica

Sistema de Facturación Electrónica UBL 2.1 • Automatización con PHPUnit 10.5

Equipo de Calidad: Apolinario, Umeres, Vasquez, Ventocilla

github.com/fujimor1/Billing